Assignment 06 solutions

COMP 271, Fall 2026

The traces below follow this reference solution. A correct solution that orders its checks differently, or stops one call earlier or later, can make a different number of calls; what must match is the value each function returns.

Rules for every problem

Every one of the six must be recursive

This is the point of the assignment, and it is checked directly on your code. In each of the six functions you write:

- **No `for` or `while` loops, and no comprehensions** (`[... for ...]` counts as a loop).
- **The function must call itself.** A method calls itself through `self`, as in `self.reverse_between(...)`.
- **One `return` statement** in a function that returns a value, and **none** in one that does not. Set a variable such as `result` in each branch and return it once at the end, the way the Recursion II deck did.
- **No built-in that walks a whole list or string for you**: `sum`, `any`, `all`, `sorted`, `.sort()`, `.count()`, `.index()`, `reversed`, `.reverse()`, or `max`/`min` of a whole list. `max` and `min` of two values, as in the Day 11 `largest`, are fine.
- **At least one comment inside each function you write.** The template's own comments do not count toward yours.
- **Do not change a list you are given.** `is_sorted` and `first_index_between` only read their input.

The two constants at the top of the template, `BASE` and `SPLIT_PARTS`, name the `10` and the `2` you will need; use them. A number counts as named only when it sits on a line of its own, like `SPLIT_PARTS: int = 2`: a line such as `mid = (lo + hi) // 2` names the midpoint, not the `2`, so that `2` is still a magic value. As always, `0`, `1` and `-1` may be written bare, and so may `2` or `-2` used directly as a position, as in `text[-2]` or `text[2:]`.

Do not Distribute

Problem 1: Index-Based Recursion in a Dynamic Array

The template's `DynamicArray` is a trimmed copy of Week03's, keeping only what this problem needs: a list `_underlying` that always holds `_capacity` slots, a count `_size` of how many slots hold real values, and the sentinel `_EMPTY` (`-1`) in every slot past the end.

On Day 9 the deck summed a list with `total(nums[1:])` and noted that each slice **copies** the rest of the list, and that passing an index instead avoids the copies. This problem makes you do exactly that. Neither method may slice: each recursive call passes along **positions**, and every call works on the same `_underlying`.

Implement two methods. The template already gives you `reverse()` and `count(value)`, which call yours on the whole array.

- `reverse_between(lo, hi)` reverses the stored values at positions `lo` through `hi` (both included), **in place**.
- `count_matches_from(value, start)` returns how many stored values at positions `start` through `len(self) - 1` equal `value`.

Example

```
a = DynamicArray.from_values([10, 20, 30, 40, 50])   # capacity 8
a.reverse_between(1, 3)
print(a)              # [10, 40, 30, 20, 50]
a.reverse()
print(a)              # [50, 20, 30, 40, 10]
a._underlying         # [50, 20, 30, 40, 10, -1, -1, -1]
```

Explanation: only positions 1 through 3 change order. The three sentinel slots past the end are never touched: only positions `0` through `len(a) - 1` hold values.

Example

```
same = a
a.reverse()
print(same)           # [10, 40, 30, 20, 50]
```

Explanation: `reverse_between` returns nothing; it changes the array itself. `same` and `a` are two names for **one** object, so `same` sees the change. For the same reason, never write `a = a.reverse()`: that would leave `a` holding `None`.

Example

```
b = DynamicArray.from_values([7, 3, 7, 7, 1])
b.count(7)                    -> 3
b.count_matches_from(7, 1)    -> 2
b.count_matches_from(7, 5)    -> 0
b.count(-1)                   -> 0
```

Explanation: starting at position 1 skips the first `7`. Starting at `len(b)` leaves nothing to count. `b` has three sentinel slots holding `-1`, but they are not stored values, so `count(-1)` is `0`.

Do not Distribute

Trace it before you run it

Once your `reverse_between` works, trace your own version on paper for `a.reverse()` on `[10, 20, 30, 40, 50]`: list every call it makes, with its `lo` and `hi`, and what the array looks like when that call starts. Then predict how many calls your version makes for an array of 6 values, and of 7, and check your prediction by adding a temporary `print`.

Constraints

- Both methods must be recursive, as described under Rules for every problem. **No slicing** anywhere in either method, and no `reversed()` or `.reverse()` on a list.
- `reverse_between` returns nothing, so it has **no** `return` statement.
- In `reverse_between`, an empty range (`lo > hi`) does nothing. A non-empty range that reaches outside positions `0` through `len(self) - 1` raises `IndexError`.
- In `count_matches_from`, a `start` at or past `len(self)` gives `0`, and a negative `start` raises `IndexError`.
- Arrays in the tests hold at most **500** values, so the number of nested calls stays well under Python's limit of about 1000.
- `week06.py` may **not** contain any `import` statements.
- The Programmer's Pact applies to everything you write.

What to Submit

Implement these two methods of `DynamicArray` in `week06.py`, keeping the signatures exactly as the template gives them:

Method	Returns
`reverse_between(self, lo: int, hi: int) -> None`	nothing; reverses positions `lo` through `hi` in place
`count_matches_from(self, value: int, start: int) -> int`	how many stored values from `start` on equal `value`

Solution: technical note

Both methods pass **positions** down the recursion instead of pieces of the list. Every call reads and writes the same `_underlying`, which is why neither needs to slice, and why neither may.

`reverse_between` is a two-pointer swap. The outermost pair trades places, and the recursive call handles everything strictly inside it: `lo + 1` to `hi - 1`. The range shrinks by two each call, so it ends in one of two base cases: `lo == hi` (odd length, one middle item left, already in place) or `lo > hi` (even length, nothing left). Both are handled by the same test, `if lo < hi:`, and in both the method simply does nothing. It returns no value, so it has no `return` at all; the change is to the array itself. The bounds check sits in front, `lo <= hi and (lo < 0 or hi >= self._size)`, so an empty range is always allowed but a real one may never reach the sentinel slots.

Trace of `a.reverse()` on `[10, 20, 30, 40, 50]`:

Call	Array when the call starts
`reverse_between(0, 4)`	`[10, 20, 30, 40, 50]`
`reverse_between(1, 3)`	`[50, 20, 30, 40, 10]`
`reverse_between(2, 2)`	`[50, 40, 30, 20, 10]` (base case: one item)

Do not Distribute

This version makes `n // 2 + 1` calls for an array of `n` values: 4 for 6 values, 4 for 7.

`count_matches_from` is the Day 9 `total` with an index. Its base case is `start >= self._size`, **not** `start >= len(self._underlying)`: stopping at capacity would walk into the sentinels, and `count(-1)` would report them. The recursive case adds `int(self._underlying[start] == value)` (a `True` counts as 1) to the count from `start + 1`.

Trace of `b.count(7)` on `[7, 3, 7, 7, 1]`, returns listed as they unwind:

Call	Returns
`count_matches_from(7, 5)`	`0` (base case)
`count_matches_from(7, 4)`	`0 + 0 = 0`
`count_matches_from(7, 3)`	`1 + 0 = 1`
`count_matches_from(7, 2)`	`1 + 1 = 2`
`count_matches_from(7, 1)`	`0 + 2 = 2`
`count_matches_from(7, 0)`	`1 + 2 = 3`

Each call adds one frame, which is why the arrays in this problem are capped at 500 values.

Solution code

```python
class DynamicArray:
    """A fixed-capacity array of non-negative integers that doubles its
    capacity whenever it fills up.
    """

    _DEFAULT_CAPACITY: int = 4
    _GROWTH_FACTOR: int = 2
    _EMPTY: int = -1  # sentinel: "this slot holds no value"
```

```python
    def __init__(self, capacity: int = _DEFAULT_CAPACITY) -> None:
        # A capacity below 1 would leave nowhere to put the first value.
        if capacity < 1:
            raise ValueError("capacity must be at least 1")
        self._capacity: int = capacity
        self._size: int = 0
        # Every slot starts as the sentinel.
        self._underlying: list[int] = []
        for _ in range(self._capacity):
            self._underlying.append(self._EMPTY)
```

```python
    @classmethod
    def from_values(cls, values: list[int]) -> "DynamicArray":
        """Build a new array holding `values`, in order."""
        result = cls()
        for value in values:
            result.add(value)
        return result
```

Do not Distribute

```
    def __str__(self) -> str:
        # Show only the filled slots -- sentinel slots are internal detail.
        inside = ""
        for i in range(self._size):
            inside = inside + str(self._underlying[i])
            if i < self._size - 1:
                inside = inside + ", "
        return "[" + inside + "]"
```

```
    def __len__(self) -> int:
        """Return the number of values stored."""
        return self._size
```

```
    def get_capacity(self) -> int:
        """Return the total number of slots, sentinels included."""
        return self._capacity
```

```
    def resize(self) -> None:
        # Build a bigger array, copy what is real, then swap it in.
        bigger_capacity = self._capacity * self._GROWTH_FACTOR
        temp: list[int] = []
        for _ in range(bigger_capacity):
            temp.append(self._EMPTY)
        for i in range(self._size):
            temp[i] = self._underlying[i]
        self._underlying = temp
        self._capacity = bigger_capacity
```

```
    def add(self, value: int) -> None:
        # A negative value could not be told apart from the sentinel.
        if value < 0:
            raise ValueError("values must be non-negative")
        # Grow first if the array is full, so index _size is always writable.
        if self._size >= self._capacity:
            self.resize()
        self._underlying[self._size] = value
        self._size = self._size + 1
```

Do not Distribute

```
    # ----- Problem 1: index-based recursion -----

    def reverse_between(self, lo: int, hi: int) -> None:
        """Reverse the stored values at positions lo through hi (both
        included), in place.

        Nothing is returned: the array itself changes. If lo > hi the range
        is empty and nothing happens. A non-empty range that reaches outside
        positions 0 through len(self) - 1 raises IndexError, so the sentinel
        slots past the end are never touched.
        """
        # A non-empty range must stay inside the stored values; the
        # sentinel slots are never touched.
        if lo <= hi and (lo < 0 or hi >= self._size):
            raise IndexError("positions must lie between 0 and len - 1")
        # Base case: an empty (lo > hi) or one-item range is already reversed.
        if lo < hi:
            # Swap the two ends ...
            temp = self._underlying[lo]
            self._underlying[lo] = self._underlying[hi]
            self._underlying[hi] = temp
            # ... then reverse what lies strictly between them.
            self.reverse_between(lo + 1, hi - 1)
```

```
    def reverse(self) -> None:
        """Given: reverse every stored value."""
        self.reverse_between(0, len(self) - 1)
```

```
    def count_matches_from(self, value: int, start: int) -> int:
        """Return how many of the stored values at positions start through
        len(self) - 1 equal `value`.

        A start at or past len(self) leaves nothing to count, so the answer
        is 0. A negative start raises IndexError. Only stored values count:
        the sentinel slots never do, even when `value` is -1.
        """
        if start < 0:
            raise IndexError("start must be non-negative")
        if start >= self._size:
            # Base case: no stored values left (the sentinels never count).
            result = 0
        else:
            # This position (True counts as 1) plus everything after it.
            here = int(self._underlying[start] == value)
            result = here + self.count_matches_from(value, start + 1)
        return result
```

```
    def count(self, value: int) -> int:
        """Given: count every stored copy of `value`."""
        return self.count_matches_from(value, 0)
```

Do Not Distribute

Test cases

```
def problem_1() -> None:
    # ----- Problem 1 -----
    a = DynamicArray.from_values([10, 20, 30, 40, 50])
    print(a, len(a), a.get_capacity())      # expected: [10, 20, 30, 40, 50] 5 8
    a.reverse_between(1, 3)
    print(a)                                # expected: [10, 40, 30, 20, 50]
    a.reverse()
    print(a)                                # expected: [50, 20, 30, 40, 10]
    # Peeking at a private attribute, only to show the sentinels stay put.
    # Code outside the class should not do this.
    print(a._underlying)                    # expected: [50, 20, 30, 40, 10, -1, -1, -1]
    a.reverse_between(2, 2)
    print(a)                                # expected: [50, 20, 30, 40, 10]
    # reverse_between changes the array itself: a second name sees it too.
    same = a
    a.reverse()
    print(same)                             # expected: [10, 40, 30, 20, 50]

    b = DynamicArray.from_values([7, 3, 7, 7, 1])
    print(b.count(7), b.count_matches_from(7, 1), b.count_matches_from(7, 4))   # expected:
    ↪  3 2 0
    print(b.count_matches_from(7, 5), b.count(-1))  # expected: 0 0
    empty = DynamicArray()
    empty.reverse()
    print(empty, empty.count(0))            # expected: [] 0
```

Do not Distribute

Problem 2: Peel One Piece per Call

Two functions, each of which removes one piece of its input per call, like `digit_sum` in the Day 9 handout and `reverse` and `is_pal` in the Day 9 deck.

- `digits_ascending(n)` returns `True` if every decimal digit of `n` is **at most** the digit to its right. Equal neighbors are fine.
- `collapse(text)` returns `text` with every run of repeated adjacent characters cut down to one.

Example

```
digits_ascending(1223)   -> True
digits_ascending(1323)   -> False
digits_ascending(7)      -> True
digits_ascending(0)      -> True
digits_ascending(10)     -> False
```

Explanation: in `1223` each digit is at most the next one; the two `2`s are equal, which is allowed. In `1323` the `3` is followed by a smaller `2`. A single digit, `0` included, has no neighbor to be out of order with.

Example

```
collapse("aabccca")      -> "abca"
collapse("mississippi")  -> "misisipi"
collapse("zzzz")         -> "z"
collapse("")             -> ""
```

Explanation: each run of the same character, however long, becomes one copy of it. The final `a` of `"aabccca"` is kept: it is not next to the `aa` at the start.

Trace it before you run it

Once your functions work, trace your own versions on paper: list every call `digits_ascending(1323)` makes and what each one returns, then do the same for `collapse("aabccca")`. Check your trace with a temporary `print` at the top of the function.

Constraints

- Both functions must be recursive, as described under Rules for every problem.
- `digits_ascending` must work on the digits through `n % BASE` and `n // BASE`. **Do not convert `n` to a string** (no `str(n)`, no f-string, no `%` or `.format` formatting). Valid inputs are `0 <= n < 10**18`; a negative `n` raises `ValueError`.
- In `collapse`, slicing is allowed. Tests use strings of at most **500** characters.
- The Programmer's Pact applies to everything you write.

What to Submit

Implement these two functions in `week06.py`:

Function	Returns
`digits_ascending(n: int) -> bool`	whether each digit is at most the next
`collapse(text: str) -> str`	`text` with each run of repeats cut to one character

Do not Distribute

Solution: technical note

`digits_ascending` peels the last digit off with `n // BASE`, just as `digit_sum` did. The difference is what each call checks: the **last two** digits, `(n // BASE) % BASE` and `n % BASE`. If the one on the left is at most the one on the right, the rest of the number still has to be checked, and that is the recursive call on `n // BASE`. Any number below `BASE` is a single digit and is in order by itself.

The combine line is `before_last <= last and digits_ascending(n // BASE)`. Python's `and` stops as soon as its left side is `False`, so the recursion stops at the first out-of-order pair:

Call	Compares	Returns
`digits_ascending(1323)`	`2 <= 3`: yes	result of the next call
`digits_ascending(132)`	`3 <= 2`: no	`False`, with no further call

Only two calls: `digits_ascending(13)` never runs. On `1223` the calls go `1223, 122, 12, 1`, and the single digit `1` is the base case that returns `True`.

`collapse` looks at the first two characters. If they are equal, the first is a repeat and is dropped: the answer is `collapse(text[1:])`. If they differ, the first starts a new run and is kept: `text[0] + collapse(text[1:])`. A string of length 0 or 1 has nothing to compare against and is returned as is.

Call	First two equal?	Returns
`collapse('aabccca')`	yes, drop	`'abca'`
`collapse('abccca')`	no, keep `a`	`'abca'`
`collapse('bccca')`	no, keep `b`	`'bca'`
`collapse('ccca')`	yes, drop	`'ca'`
`collapse('cca')`	yes, drop	`'ca'`
`collapse('ca')`	no, keep `c`	`'ca'`
`collapse('a')`	base case	`'a'`

The last character of each run is the one that survives, since it is the one followed by a different character, or by nothing at all.

Do not Distribute

Solution code

```
BASE: int = 10  # peel the last decimal digit: n % BASE, n // BASE

# ----- Problem 2: peel one piece per call -----


def digits_ascending(n: int) -> bool:
    """Return True if every decimal digit of n is at most the digit to its
    right.

    Equal neighbors are fine: 1223 gives True, 1323 gives False. A single
    digit gives True. A negative n raises ValueError. Work with the digits
    through n % BASE and n // BASE; do not convert n to a string.
    """
    if n < 0:
        raise ValueError("n must be non-negative")
    if n < BASE:
        # Base case: a single digit is in order by itself.
        result = True
    else:
        # Compare the last two digits, then check the rest without the last.
        last = n % BASE
        before_last = (n // BASE) % BASE
        result = before_last <= last and digits_ascending(n // BASE)
    return result
```

```
def collapse(text: str) -> str:
    """Return text with every run of repeated adjacent characters cut down
    to one: "aabccca" gives "abca".
    """
    if len(text) <= 1:
        # Base case: nothing to compare against.
        result = text
    elif text[0] == text[1]:
        # The head repeats the next character: drop it.
        result = collapse(text[1:])
    else:
        # The head starts a new run: keep it.
        result = text[0] + collapse(text[1:])
    return result
```

Test cases

```
def problem_2() -> None:
    # ----- Problem 2 -----
    print(digits_ascending(1223), digits_ascending(1323), digits_ascending(7))
    # expected: True False True
    print(digits_ascending(0), digits_ascending(10), digits_ascending(59))
    # expected: True False True
    print(collapse("aabccca"), collapse("mississippi"))   # expected: abca misisipi
    print(repr(collapse("")), collapse("zzzz"))            # expected: '' z
```

Do not Distribute

Problem 3: Divide and Conquer

On Day 11 and Day 12 we wrote `count_letter`, `reverse`, `largest` and `remove_blanks` the same way: split the input at its midpoint, solve each half with a recursive call, and combine the two answers. Only the base case and the combine step changed from one function to the next.

Write `is_sorted(values)`, which returns `True` if `values` is in non-decreasing order (each item at most the next), in that same divide, conquer, combine shape.

Example

```
is_sorted([2, 5, 5, 8])          -> True
is_sorted([])                    -> True
is_sorted([7])                   -> True
is_sorted([1, 3, 2, 4])          -> False
is_sorted([1, 2, 5, 3, 4, 6])    -> False
```

Explanation: equal neighbors are allowed, and zero or one item is always in order.

Example

```
is_sorted(list(range(5_000)))    -> True
```

Explanation: 5,000 items is far more than Python's limit of about 1000 nested calls. A version that peels off one item per call hits `RecursionError` here. Halving the list each call needs only 14 levels. This line is in the template's test block, so you will see it fail on your own machine if your recursion does not halve.

Trace it before you run it

Once your `is_sorted` works, draw its call tree for `is_sorted([1, 2, 5, 3, 4, 6])`, the way the Day 11 deck drew `count_letter('cheese')`, and write what each call returns.

Constraints

- `is_sorted` must be recursive, as described under Rules for every problem, and must **split its input in two at the midpoint** on each call.
- `is_sorted` must not change `values`.
- No `sorted()` and no `.sort()`: comparing against a sorted copy is not divide and conquer.
- Slicing is allowed. Tests use lists of up to **100,000** values.
- The Programmer's Pact applies to everything you write.

What to Submit

Function	Returns
`is_sorted(values: list[int]) -> bool`	whether `values` is in non-decreasing order

Do not Distribute

Solution: technical note

The divide and conquer steps are the same as in class: split at `middle = len(values) // SPLIT_PARTS` and call `is_sorted` on each half. The combine step is where this problem differs from `count_letter` and `largest`. Two sorted halves do not make a sorted list: `[1, 2, 5]` and `[3, 4, 6]` are each in order, but `5` comes before `3`. The whole list is sorted only if both halves are **and** the seam is in order (the last item of the left half is at most the first item of the right): `values[middle - 1] <= values[middle]`.

Call tree for `is_sorted([1, 2, 5, 3, 4, 6])`:

Call	Returns
`is_sorted([1, 2, 5, 3, 4, 6])`	`True and True and 5 <= 3 = False`
`is_sorted([1, 2, 5])`	`True and True and 1 <= 2 = True`
`is_sorted([3, 4, 6])`	`True and True and 3 <= 4 = True`
one- and two-item calls below	all `True`

The `False` comes from the seam check in the very first call, after both halves have come back `True`.

Halving is what lets this run on 100,000 values: the depth is 18 calls, where peeling one item per call would need 100,000 and hit `RecursionError` near 1000.

Solution code

```
SPLIT_PARTS: int = 2  # halves per split

# ----- Problem 3: divide and conquer -----


def is_sorted(values: list[int]) -> bool:
    """Return True if values is in non-decreasing order (each item at most
    the next).

    Divide and conquer: split at the midpoint, check each half, then
    combine the two answers. Do not change `values`.
    """
    if len(values) <= 1:
        # Base case: zero or one item is already in order.
        result = True
    else:
        # Divide: split at the midpoint.
        middle = len(values) // SPLIT_PARTS
        # Conquer: check each half.
        left = is_sorted(values[:middle])
        right = is_sorted(values[middle:])
        # Combine: both halves sorted, and the seam between them in order.
        result = left and right and values[middle - 1] <= values[middle]
    return result
```

Do not Distribute

Test cases

```
def problem_3() -> None:
    # ----- Problem 3 -----
    print(is_sorted([2, 5, 5, 8]), is_sorted([]), is_sorted([7]))
    # expected: True True True
    print(is_sorted([1, 3, 2, 4]), is_sorted([1, 2, 5, 3, 4, 6]))
    # expected: False False
    # Too deep to peel one item per call: only halving gets through.
    print(is_sorted(list(range(5_000))))    # expected: True
```

Do not Distribute

Problem 4: Binary Search for the First Match

The Day 9 deck's recursive `find` searched a sorted list by checking the middle and throwing away the half that could not hold the target. It assumed every value appeared once. Here the list may hold **repeats**, and you must return the position of the **first** one.

Write `first_index_between(nums, target, lo, hi)`, which returns the smallest index `i` with `lo <= i <= hi` and `nums[i] == target`, or `-1` if there is none. The template gives you `find_first(nums, target)`, which calls yours on the whole list.

Example

```
nums = [2, 5, 5, 5, 8, 12, 12, 23]
find_first(nums, 5)     -> 1
find_first(nums, 12)    -> 5
find_first(nums, 2)     -> 0
find_first(nums, 7)     -> -1
find_first([], 5)       -> -1
```

Explanation: 5 sits at positions 1, 2 and 3; the answer is the first of them.

Example

```
first_index_between(nums, 5, 2, 7)    -> 2
first_index_between(nums, 5, 4, 7)    -> -1
find_first([4] * 100_000, 4)          -> 0
find_first(list(range(100_000)), 99_999)    -> 99999
```

Explanation: only positions `lo` through `hi` count, so a 5 before position 2 does not. The last two lines are there to make sure your search really halves its range: halving needs about 17 nested calls on 100,000 values, while stepping one position at a time needs up to 100,000 and runs past Python's limit of about 1000. Both lines are in the template's test block, so you will see this on your own machine.

Trace it before you run it

Once your function works, trace your own version on paper: list the `lo` and `hi` of every call `find_first(nums, 5)` makes on the list above, and what each call returns.

Constraints

- `first_index_between` must be recursive, as described under Rules for every problem, and must **halve** the range `lo` to `hi` on each call. Tests use lists of up to **100,000** values.
- `nums` is sorted in non-decreasing order. Callers always pass `0 <= lo` and `hi < len(nums)`, or an empty range with `lo > hi`.
- No slicing: pass positions, as `find` did. `first_index_between` must not change `nums`.
- The Programmer's Pact applies to everything you write.

Do not Distribute

What to Submit

Function	Returns
`first_index_between(nums: list[int], target: int, lo: int, hi: int) -> int`	the first position of `target` in `lo..hi`, or `-1`

Solution: technical note

The first two branches are `find` from Day 9: if the middle value is too small, the target can only be to the right (`mid + 1` to `hi`); if it is too big, only to the left (`lo` to `mid - 1`). Either way the range halves.

The new case is a match. With repeats allowed, `mid` holds the target but an earlier copy may sit to its left, so the function searches `lo` to `mid - 1` anyway. If that search finds something, it is earlier and it wins; if it returns `-1`, then `mid` is the first. An empty range (`lo > hi`) is the base case and returns `-1`. The range still halves on every call, match or not, so the depth stays near `log2(n)`.

Trace of `find_first([2, 5, 5, 5, 8, 12, 12, 23], 5)`:

Call	`mid`, `nums[mid]`	Returns
`first_index_between(nums, 5, 0, 7)`	3, 5: match, look left	1
`first_index_between(nums, 5, 0, 2)`	1, 5: match, look left	1
`first_index_between(nums, 5, 0, 0)`	0, 2: too small, go right	-1
`first_index_between(nums, 5, 1, 0)`	empty range	-1 (base case)

The first call finds a 5 immediately, at position 3, but the answer comes from the second call. The last two calls confirm that nothing earlier exists.

Do not Distribute

Solution code

```
SPLIT_PARTS: int = 2  # halves per split

# ----- Problem 4: first match by binary search -----


def first_index_between(nums: list[int], target: int, lo: int, hi: int) -> int:
    """Return the smallest index i with lo <= i <= hi and nums[i] == target,
    or -1 if there is none.

    nums is sorted in non-decreasing order and may hold repeats. Callers
    guarantee 0 <= lo and hi < len(nums), or lo > hi (an empty range).
    Halve the range on every call, as find did in class. Do not change
    `nums`.
    """
    if lo > hi:
        # Base case: an empty range holds nothing.
        result = -1
    else:
        mid = (lo + hi) // SPLIT_PARTS
        if nums[mid] < target:
            # Everything up to mid is too small: search the right part.
            result = first_index_between(nums, target, mid + 1, hi)
        elif nums[mid] > target:
            # Everything from mid on is too big: search the left part.
            result = first_index_between(nums, target, lo, mid - 1)
        else:
            # A match, but an earlier copy may sit to the left.
            earlier = first_index_between(nums, target, lo, mid - 1)
            result = mid
            if earlier != -1:
                result = earlier
    return result
```

```
def find_first(nums: list[int], target: int) -> int:
    """Given: search the whole list."""
    return first_index_between(nums, target, 0, len(nums) - 1)
```

Test cases

```
def problem_4() -> None:
    # ----- Problem 4 -----
    nums = [2, 5, 5, 5, 8, 12, 12, 23]
    print(find_first(nums, 5), find_first(nums, 12), find_first(nums, 2))
    # expected: 1 5 0
    print(find_first(nums, 7), find_first(nums, 99), find_first([], 5))
    # expected: -1 -1 -1
    print(first_index_between(nums, 5, 2, 7), first_index_between(nums, 5, 4, 7))
    # expected: 2 -1
    print(find_first([4] * 100_000, 4))     # expected: 0
    print(find_first(list(range(100_000)), 99_999))   # expected: 99999
```

Do not Distribute